

IC526 Advanced Compilers

Lec 01: Introduction

Yoonseung Kim

DGIST EECS

Fall 2026

Instructor: Yoonseung Kim (김윤승)

- Assistant Professor, EECS Dept. (Nov 2025 – Current)

Website: <https://dgist-vesl.github.io>

Contact: yoonseung.kim@dgist.ac.kr

- Research Area

- Programming Languages

- └ Software Verification

- └ Compilers

- └ Real-Time Systems

- └ Distributed Algorithms



COMP CERT

A Formally Verified C Compiler



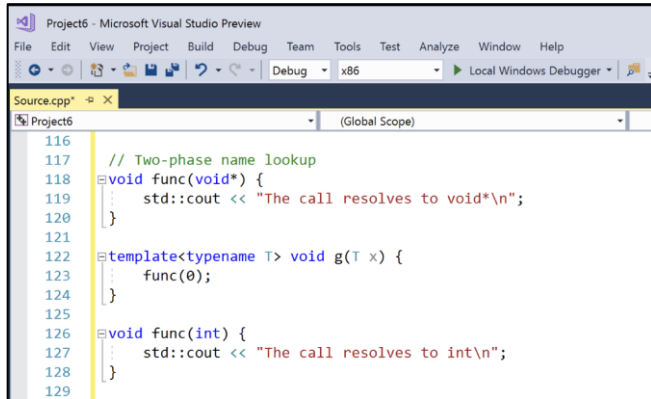
A Mainstream Compiler

Compilers: A Quick Overview

<https://github.com/DGIST-VerifiedSystemsGroup/AdvComp2026f>

What Compilers Do: An Analogy

High-level idea in any representation



```
116 // Two-phase name lookup
117 void func(void*) {
118     std::cout << "The call resolves to void*\n";
119 }
120
121 template<typename T> void g(T x) {
122     func(0);
123 }
124
125 void func(int) {
126     std::cout << "The call resolves to int\n";
127 }
128
129
```

```
#include <stdio.h>

int main() {
    printf("Hello, World!\n");

    return 0;
}
```

[Translation]

```
main:
.LFB0:
.cfi_startproc
endbr64
pushq   %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq    %rsp, %rbp
.cfi_def_cfa_register 6
leaq    .LC0(%rip), %rax
movq    %rax, %rdi
call    puts@PLT
```

A fixed set of instructions



Combo attacks



You (Brain-to-fingers)



Compilers Bridge Different Representations

Programming Languages

```
#include <stdio.h>

int main() {
    printf("Hello, World!\n");

    return 0;
}
```

Focus: Express complex ideas precisely

- Writability
- Readability
- Safety

Compiler

Instruction Set Architecture (ISA)

```
main:
.LFB0:
    .cfi_startproc
    endbr64
    pushq    %rbp
    .cfi_def_cfa_offset 16
    .cfi_offset 6, -16
    movq     %rsp, %rbp
    .cfi_def_cfa_register 6
    leaq     .LC0(%rip), %rax
    movq     %rax, %rdi
    call     puts@PLT
```

Focus: Manipulate HW resources

- Generality
- High performance
- Power efficiency

Why Learn Compilers?

- **A skill in demand**

Efficiency is money

Programmer productivity,
compile time, runtime resources
— 1% faster → billions saved.

New tech keeps coming

New hardware (e.g., AI-chips),
new language concepts
— each needs a new compiler.

- **Integrated Understanding**

Programming Languages

- Language design
- Translation correctness

Compilers

Systems

- Low-level control
- Memory layout

Computer Architecture

- Machine instructions
- Pipelining

Course Overview

Goals / Non-Goals



Goals

Working Knowledge

Get hands-on with every stage, using the available tools to build a compiler of your own.

Correctness Reasoning

Know what each stage guarantees the next and argue why a transformation preserves meaning.

Field Literacy

Cover enough topics to recognize the terms and ideas when you meet them again.



Non-Goals (≠ unimportant)

Theory from the Ground Up

Formal languages and automata, parsing theory, LR(k), ...

From-Scratch Implementation

Lexer, parser, IR design, machine code generation, ...

A Full Catalog of Optimizations

Constant folding, loop unrolling, inlining, alias analysis, ...

What This Course Aims To Offer

Compiler Newbies

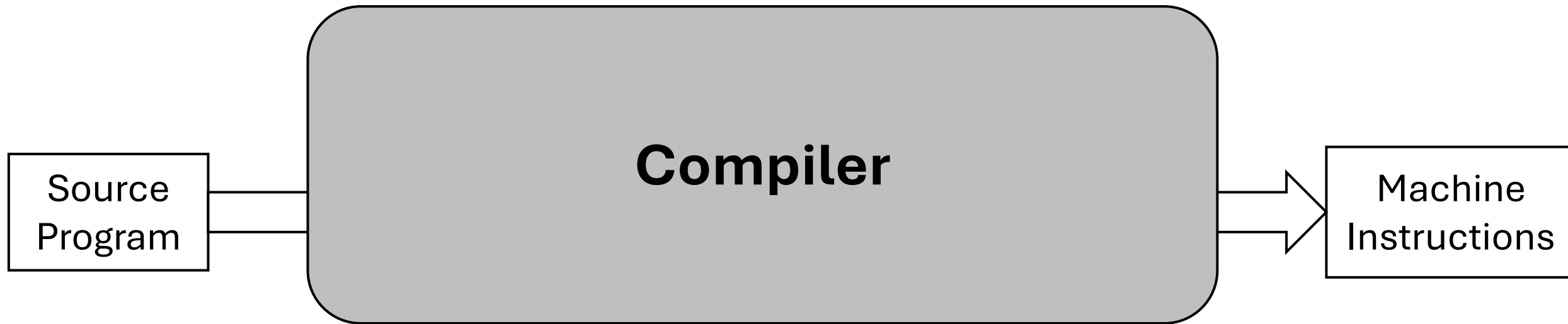
- Understanding of the overall structure of compilers
- Necessary theories
- Get working knowledge:
how to develop a prototype compiler

Compiler Pros

- How to manage an LLVM project
- Understanding of basic Rust
- Experience of handling
a fancy language feature in compilers

Course Design

- We learn the principle along with constructing a full compiler



(A subset of) Rust

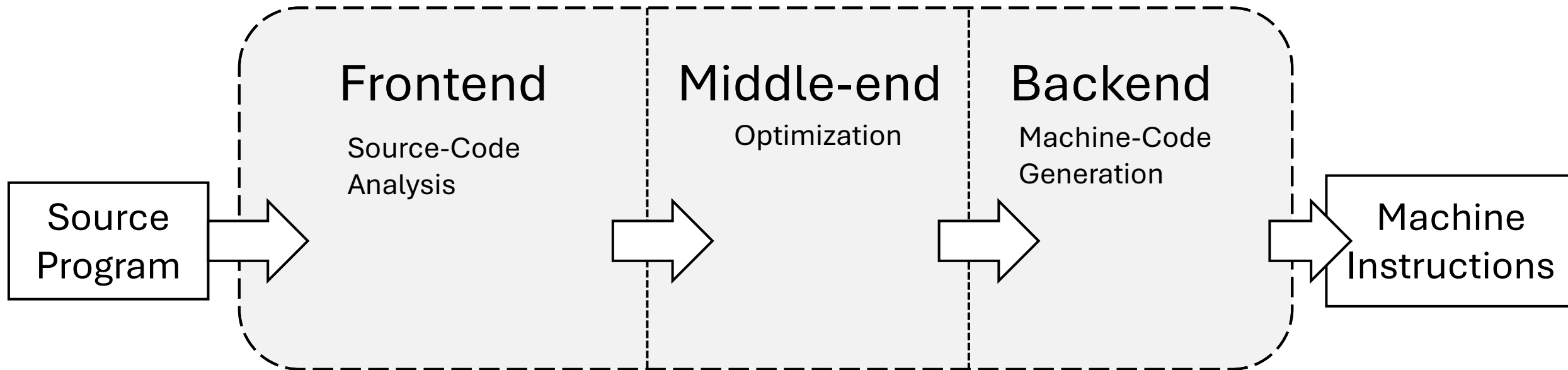
- Most-loved language
- Safe resource management



- Open ISA
- Simple, based on RISC

Course Design

- We learn the principle along with constructing a full compiler



(A subset of) Rust

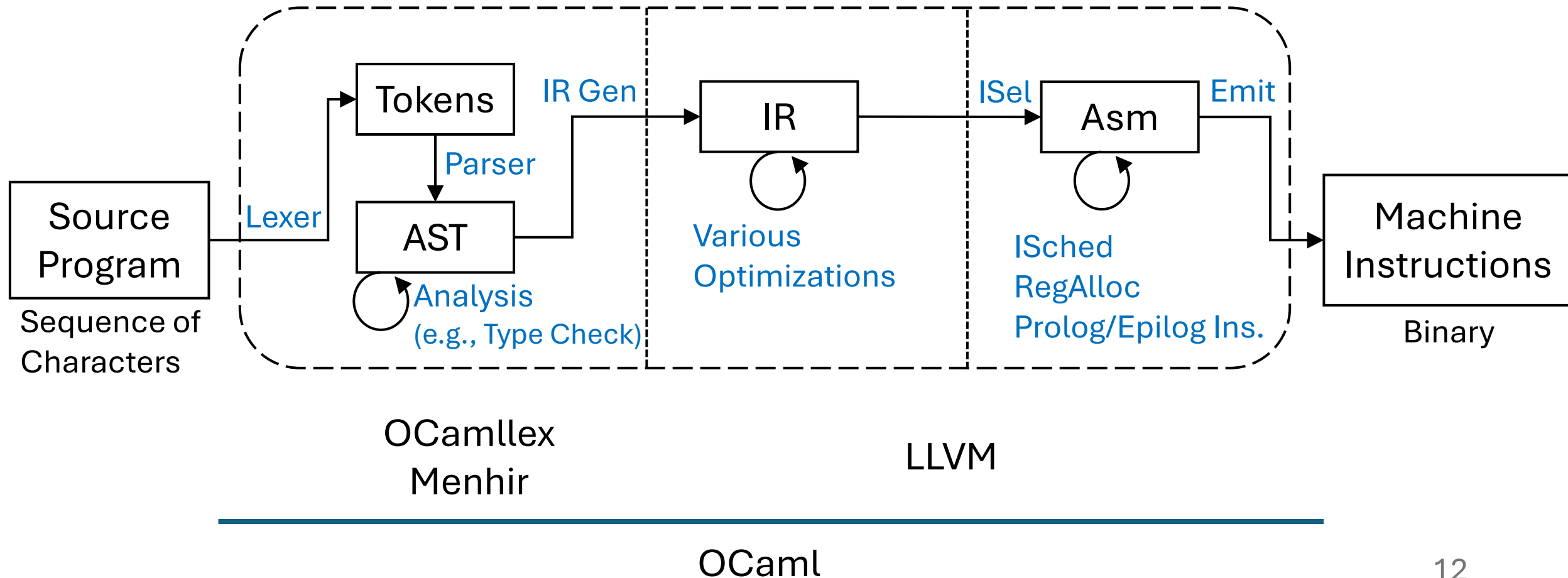
- Most-loved language
- Safe resource management



- Open ISA
- Simple, based on RISC

Course Design

- We learn the principle along with constructing a full compiler



Course Plan (Subject to Change)

- Intro
- OCaml
- Rust
- Automata

- Lexer
- Parser
- Type Check
- LLVM IR
- IRGen

PA1 (Warm-Up)

PA2 (Lexer & Parser)

PA3 (Type Check & IR Generation)

W01
(8/24-26)

W02
(8/31-9/2)

W03
(9/7-9)

W04
(9/14-16)

W05
(9/21-23)

W06
(9/28-30)

W07
(10/7)

Midterm
(10/12?)

- SSA – Mem2Reg
- Optimizations
- Correctness Reasoning

- Backend
- Linker

- Project Presentation

PA4 (Optimization)

Individual Project

W09
(10/19-21)

W10
(10/26-28)

W11
(11/2-4)

W12
(11/9-11)

W13
(11/16-18)

W14
(11/23-25)

W15
(11/30-12/2)

Final
(Report)

Course Logistics

- **Course Website** (**check it regularly!**)
 - <https://github.com/DGIST-VerifiedSystemsGroup/AdvComp2026f>
- **Course Materials**
 - Slides
- **Evaluation (Tentative)**
 - Midterm (40%), Assignments (20%), Project (30%), Attendance and participation (10%) – Help improve the material
- **AI & Collaboration Policy**
 - You may consult AI tools and work with your classmates.
 - Use your own judgment (it's not about your grade, but what you gained).
- **For everything else, check the course website.**

Prerequisites

- **You are expected to**
 - Have taken CS101 (in C, C++, Python, Java, etc.)
 - Understand memory references (C pointers / Python object / etc.)
 - Know how computers work with memory, register, and CPU
 - Know Git, and have a GitHub account
- **What is *not* assumed** – Everything you need will be covered in class.
 - Automata theory
 - Programming language theory
 - Undergraduate compiler course
 - OCaml or any functional language experience

Before Next Class

- **Set up your programming environment**
 - Install OPAM
 - Install the required packages via OPAM
 - Configure your development environment
 - **Get a head start on OCaml**
 - Read the lec02 slides and try the exercises (HW0).
- > **Bring your laptop with the environment already set up**